

bdelf BELF / RELF：设计与实现说明

bdelf 模型文档

2026 年 8 月 27 日

摘要

本文档完整说明 bdelf 中两族半自回归连续语言模型 BELF 与 RELF 的动机、流几何、时间场、损失、CFG、训练接口与推理循环。二者实现上可共享模块代码，但须分别描述、分别训练，且不共享参数。共用 ELF 约定的 rectified flow ($t = 1$ 为干净端)、token 对齐 VAE、2L pack、去噪器 G ，以及 G 之后的出口读 token。流空间是入口输出维 $X = \text{latent_dim}$ (须与 artifact 一致)； $D = \text{n_embd}$ 只是 G 隐层。默认出口为 $\hat{x}_0 \in X$ 再走 VAE-dec (Cola)；对照从隐状态 $D \rightarrow V$ 直接出 logits (ELF)。默认 CE 对 G 反向传播。**BELF** 在块内未知槽共享同一标量 t ，已知余数钉干净条件，块间将提交结果写入 KV cache。**RELF** 在窗内使用降序局部时间梯子，每步自窗左端移出 (pop) S 个槽；当前窗已知余数钉干净条件。每一族、每一档 `latent_tune` 均须完成主训与扩展 token 预算；生成长度上限为 `max_seq_len`。工程路径见同目录上级 `README.md`。本文档不冻结 GPU 机时额度，亦不提供训练结论。

1 动机与定位

ELF [1] 表明：在冻结上下文嵌入上，以 $t = 1$ 为干净端做线性插值，并由 x-pred 转换为速度均方误差，可在连续状态空间上完成观测恢复。其生成分解为整句共享一个标量 t 、注意力全程双向，因而既无 KV cache，也不能按块提交或作可变长增量生成。Cola [2] 与 AURORA-LM [3] 采用另一分解：冻结或先训练可解码 latent，再以块因果 Flow Matching 作为 prior，由独立 decoder 读出 token。该分解已是半自回归，但块内仍共享同一时间；Cola 的时间轴与 ELF 相反 ($t = 0$ 为干净端)，其主要主张为压缩语义 latent。

因此给出两条可分别训练与评测的半自回归 ELF：BELF 在块内未知槽共享单一时间（已知余数钉干净），RELF 在窗内使用位置相关的降序时间梯子（已知余数同样钉干净）。本文档中 VAE 将因果编码器输出映射为近似各向同性、可叠加高斯噪声的 token 对齐 latent，且不缩短序列长度。下文先陈述两族共用部分，再单开时间调度章，然后分章给出 BELF 与 RELF。本文档不设定 GPU 机时上限（5090 与 A100 均未冻结额度）。

2 相关工作

本节陈述近邻工作的机制及其与本规格的差异。不以 full-width latent 本身作为设计贡献。BELF 对照 AURORA、BD3-LM 与 Cola 的块因果分解；RELF 对照 Rolling Diffusion 的局部时间场，并采用 ELF 的流几何。两族分别陈述、分别训练。

全程一个 t 的观测恢复。 ELF 冻结 T5 嵌入，采用 rectified flow，并由 x-pred 转换为速度均方误差。全序列双向、整句一个标量 t 。CoDAR [4] 将 rounding 外包给独立自回归 decoder，但仍是全序列连续扩散。二者均无已提交前缀进入 KV cache。

冻 AE + 块因果连续 prior。 Cola 将 $p(x, z_0) = p_\theta(x | z_0) p_\psi(z_0)$ 分解，并以 2L pack 一次输入干净前缀与当前块噪声。AURORA 冻结 Query-AE，对 full-width 可解码 latent 做块因果 Flow Matching。与 BELF 最近的近邻是：冻结编码器、块因果注意力、独立 decoder。BELF 当前块内已知余数采用 Cola 的 clean 条件：钉干净、只去噪未知槽。差异在于自编码器构造、流匹配配方，以及 Cola 不具备窗内降序时间梯子。

离散块同步 t 。 BD3-LM [5] 块间自回归、块内吸收态 mask、一块一个 t_b 、同一网络出 logits。这是 BELF 在离散观察空间上的对应物。本规格的状态是连续 VAE 码，token 由独立出口读出。

视频里的局部时间滑窗。 Rolling Diffusion [6] 将全局 t 映射为帧局部 t_k 。RELF 的局部时间场在滑窗后保持时间半群一致性，其约束源于该工作。其对象为视频或物理场，而非语言模型，亦无 KV cache 与 token decoder。

3 规格声明与设计要点

3.1 模型定义

在由 `latent_model+tag` 指定的 token 对齐隐空间上，以 ELF 的 rectified flow ($t = 1$ 为干净端) 训练去噪器 G ，再经出口读出 token。**BELF** 以块内未知槽同步 t 做半自回归，当前块已知余数钉干净条件；**RELF** 以窗内降序梯子逐步 pop 做滚动半自回归，当前窗已知余数钉干净条件。该分解规定：两族均在连续隐空间上沿该流做观测恢复，并借助已提交前缀的 KV cache 生成至 `max_seq_len`。实现与评测以该分解为准；本文档不预设训练结论。

参数分模型、训练、推理三类，见第 4 节。两族分开训练、分开评测。无并列的次要规格分支。

3.2 设计要点

1. BELF：将同一流用于块因果 2L/KV；未知槽共享同一 t ，块间提交 $\hat{x}_0$ ；已知余数钉干净条件；仅最后一跳对未知槽计算 CE。
2. RELF：将 ELF 流几何用于滚动窗；局部时间梯子与 pop 对齐 CE。整窗 F ，BOS 前 / EOS 后截断。已知余数钉干净条件。每步 pop S 个槽。
3. 训练日程（非第三条机制贡献）：每一族 $\times$ 每一 `latent_tune` 完成主训后再做扩展；切分见附录 A。预算与 `max_seq_len` 见表。

4 共性

两族共用加载、映射、 G 、出口、损失与训练日程。时间场见第 5 节。读出位置与生成循环见本族章。CFG 见第 6 节；提交与采样器见第 9 节。参数表按模型 / 训练 / 推理分栏；某类无字段则省略该栏。评测长度见附录 A。

4.1 参数三类

超参数划分为模型、训练、推理三类。`latent_model` 与 `tag` 属模型类：二者指定入口自编码器，因而决定入口结构，不再单列「加载」类。

模型。 网络结构。必填 `latent_model` 与 `tag`，无默认，用于调用已有加载器 `load_latent_artifact`。入口 encoder / decoder 在参数表里只记这两项与流维 `latent_dim` (X ，须等于 artifact 输出维)；词表、VAE 层数、s1 系数以加载结果为准，不列表。入口 encoder 的注意力块长以加载结果为准，不随本规格改写。`block_size` 仅 BELF: G 去噪块长，100m 默认 16，主跑 $W = T$ 。加载入口块长须为 1 (逐 token 因果) 或等于本题 BELF W 。可训档 (`full` / `mid`) 另要求入口块长为 1；块因果入口只允许 `frozen`。RELF 无此字段；加载入口块长必须为 1；窗长 `window_size` 与入口无关。

`n_embd` (记 D) 是 G 的隐层宽度，与 X 无关，须整除 `n_head`，全文只声明一次。`max_seq_len` 是位置编码与生成长度上限。出口无层数键：默认 `decoder` (Cola) 时 $\hat{x}_0$ 已在 X ，直接走加载的 VAE-dec 得 logits；对照 `linear` (ELF) 从隐状态 $D \rightarrow V$ 得 logits。两类只改 $\hat{x}_0 \rightarrow \text{logits}$ 这一段；CE 施加于哪些位置由本族规定。

`sc_cfg` 决定网络是否包含 ScaleEmbedder 与 self-cond 通道，因此进入模型指纹。

训练。 加载之后的优化规则。`latent_tune` 只改变梯度是否反向传播到 latent 本体，不改变加载身份。

<code>latent_tune</code>	含义
<code>frozen</code>	全程冻结。完整 latent 参数仍写入 s2 checkpoint。
<code>full</code>	自 step 0 起更新 latent。
<code>mid</code>	默认。已消费 token 未到 <code>latent_thaw_tokens</code> 时与冻结相同；到达该点后解冻一次，此后与 <code>full</code> 相同。解冻后为新可训参数分配新的优化器状态。

推理。 同一 checkpoint 上的采样规则。提交、采样器与 CFG 尺度见第 9 节与第 6 节；不进入训练指纹。BELF 生成使用 `block_generate`，RELF 生成使用 `rolling_generate`。

第二阶段 (s2) 始终训练映射、 G 与出口。冻结档不计算 $\mathcal{L}_{s1}$ 。`full` 与解冻后的 `mid`：只要更新 latent 的任一侧 (encoder、VAE-dec、或两侧)，就必须计算完整的 $\mathcal{L}_{s1}$ (重建 + $\text{KL}(q||N(0, I))$ + BERT-mask + ref-KL)。 $\mathcal{L}_{s1}$ 与 $\mathcal{L}$ 分开相加。出口 CE 计入 $\mathcal{L}$ 。`exit=decoder` 时 CE 经已在 X 的 $\hat{x}_0$ 与 VAE-dec 回传；VAE-dec 参数是否更新仍只由 `latent_tune` 决定 (frozen 时参数冻结，梯度仍可回到 G)。可训档的 $\mathcal{L}_{s1}$ 仍经同一 VAE-dec。`exit=decoder` 或可训档启动时须具备 VAE-dec。`mid` 解冻前仍不计算 $\mathcal{L}_{s1}$ 。速度误差仍可反向传播到 encoder；2L 左段的 `sg` 只切断 pack 左半，不切断该梯度。默认 CE 对 G 反向传播。

启动校验。 加载结果的 X 须等于配置的 `latent_dim`；入口 encoder 块长以加载结果为准，不随 W 改写。BELF：加载入口 `encoder.block_size` $\in \{1, W\}$ 。RELF 无 `block_size`；加载入口块长必须为 1，且 $S \cdot T = W$ 。可训档（full / mid）要求入口块长为 1，否则报错拒绝。两族 `time_step` $T \geq 4$ ，否则报错。 G 的宽度等于 D ；`exit` 与实际模块匹配；`exit=decoder` 或可训档须具备 VAE-dec。各族额外校验见本族章。

4.2 流、去噪器与出口

时间： $t = 1$ 干净、 $t = 0$ 纯噪。流与 v-MSE 在入口输出维 $X = \text{latent_dim}$ （须与 artifact 一致）； $D = \text{n_embd}$ 只是 G 隐层。白化留在 X ，茎 $\text{Linear}(X \rightarrow D)$ ，`FinalLayer` 回到 X ：

$$z_t = t x_0 + (1 - t) \varepsilon, \quad \hat{v} = \frac{\hat{x}_0 - z_t}{\max(1 - t, \varepsilon_t)}, \quad v^* = x_0 - \varepsilon.$$

G 输出干净点估计 $\hat{x}_0 \in \mathbb{R}^X$ ，再转换为速度并计算 v-MSE。分母取 $\max(1 - t, \varepsilon_t)$ ，其中 $\varepsilon_t = \text{vel_eps}$ ，以避免 $t \rightarrow 1$ 时除零。干净端默认取后验样本（对照均值）；推理写入 KV 的对象必须与该干净端处于同一空间 X 。

G 是一份网络，不含 ELF `decoder_prob` 双模式。条件通道为 AdaLN-Zero (DiT)，不用 ELF 的 in-context control tokens (`num_time_tokens=0`)。块内 LayerNorm 无仿射；每层调制为 $\text{SiLU} \circ \text{Linear} : D \rightarrow 6D$ ，拆成 attn / MLP 的 shift、scale、gate；残差 $x \leftarrow x + \text{gate} \cdot \text{sub}(\dots)$ 。调制末层权重与偏置零初始化，起步每块为恒等。`FinalLayer` 同样 AdaLN-Zero（调制 $D \rightarrow 2D$ 的 shift / scale），输出线性零初始化。骨干其余不变：RoPE、qk-RMSNorm、SwiGLU。此为实现默认，不列入设计要点、不列入消融。每一层 AdaLN-Zero 逐列以 (t, w_{sc}, m) 为条件（ m 进入模型指纹；不是序列 token，不改变 2L / 注意力）：未知 denoise 列 $m = \text{denoise}$ ，`sc_cfg` 为真时再以 w_{sc} 为条件；未知 decode 列 $m = \text{decode}$ ，不以 w_{sc} 为条件；左段 / 已知 / PAD 不加 m 、不以 w_{sc} 为条件。BOS 前、EOS 后的格不在当前窗，同样不加 m 。读 token 仅发生在 G 之后的出口。流与出口按槽位拆开：denoise 列只计算 v-MSE，decode 列只计算 CE。两项各自对有效位置取平均，再以 λ 相加。左段与已知槽不进入 $\mathcal{L}$ 。BELF 不引入 m_d/m_c （用跳号拆）。RELF 使用 m_d/m_c ，且 $m_c = 1$ 则 $m_d = 0$ 。注意力与 sc 通道见本族掩码节。

流与出口：

$$\mathcal{L} = \lambda_{\text{mse}} \mathcal{L}_{\text{mse}} + \lambda_{\text{ce}} \mathcal{L}_{\text{ce}}.$$

本步无读出位置时 $\mathcal{L}_{\text{ce}} = 0$ 。逐 token 的 $\|\cdot\|^2$ 对 D 取平均。BELF： $i = 0, \dots, T-2$ 时 $\mathcal{L}_{\text{mse}}$ 为右段未知有效位平均、 $\mathcal{L}_{\text{ce}} = 0$ ； $i = T-1$ 时 $\mathcal{L}_{\text{ce}}$ 为右段未知有效位 CE 平均、 $\mathcal{L}_{\text{mse}} = 0$ 。已知槽不进入 $\mathcal{L}$ 。RELF 的两项按 m_d/m_c 平均，公式见本族掩码节。默认 `exit=decoder`、`ce_detach_g=false`：CE 对 G 反向传播。 v_{tgt} 是否被 CFG 修正、AdaLN 是否以 w_{sc} 为条件，见第 6 节。可训 latent 时总目标为 $\mathcal{L} + \mathcal{L}_{\text{s1}}$ ：

$$\mathcal{L}_{\text{s1}} = \lambda_{\text{vae}} \text{CE}_{\text{rec}} + \beta \text{KL}(q_\phi \| N(0, I)) + \lambda_{\text{mask}} \mathcal{L}_{\text{mask}} + \lambda_{\text{ref}} \text{KL}(q_\phi \| q_{\phi_{\text{ref}}}).$$

CE_{rec} 与 BERT-mask 都经 VAE-dec（不经 `ExitMap`）。 $q_{\phi_{\text{ref}}}$ 是 s2 开始时加载器给出的 encoder 冻结副本。`frozen` 时 $\mathcal{L}_{\text{s1}} = 0$ 。

网络默认分为三段：加载器编码、去噪器 G 、出口。 G 一律使用逐位置时间嵌入与 AdaLN-Zero；若本族各位置 t 相同，则广播同一标量。

```

tokens
| load_latent_artifact(latent_model, tag)
| Enc (block-causal) -> z in  $R^{\{S \times X\}}$ 
| optional whiten on X
| G stem: Linear X -> D
v
pack_2l [sg(h_left)|h_t]
+ per-column (t, [w_sc if denoise], [m if unknown])
-> AdaLN-Zero
| G: n_layer x (AdaLN-Zero -> Attn(RoPE, qk-RMSNorm)
|               -> AdaLN-Zero -> SwiGLU)
| FinalLayer (AdaLN-Zero) D -> X -> x0_hat
v
Exit: x0_hat in X + VAE-dec | hidden D -> V

```

出口叠法。 出口无层数键。`linear` (ELF): 从 G 隐状态 $\text{Linear}(D \rightarrow V)$ 出 logits。`decoder` (Cola): $\hat{x}_0$ 已在流空间 X , 直接调用加载的 VAE-dec。VAE-dec 的层数 / 块长以加载结果为准, 不在本规格另建。CE 施加于哪些位置仍由本族规定。

喂给出口的序列两族不同。BELF 训练只把右段 $\hat{x}_0$ (一批块拼成一条) 送进出口, 不拼接已提交前缀; 推理 `block_generate` 把已提交的干净码与当前块 $\hat{x}_0$ 拼接后再取末 W 个 logits。RELF 训练在 `decoder` 下对每个窗做 $\text{cat}(\hat{x}_0^{\text{left}}[:g], \hat{x}_0^{\text{win}})$ ($g = u + k_0$), 再取窗内 logits; `linear` 只看右段。推理 `rolling_generate` 把 G 的整段 2L 输出送进出口, 再切右段窗。因此: BELF 训练出口看不到前缀, 推理前缀是提交码而不是 G 左段输出; RELF 训练与推理的 `decoder` 前缀都是 G 左段的 $\hat{x}_0$ 。

2L 与 Cola 块因果结构同构: 已完成段写入 KV 后组内双向、组间单向; 右段可见左段。BELF 的组是本规格 `block_size` W ; RELF 已完成段按加载入口块长 (必须为 1, 即逐 token 因果), 窗内组是档 S 。左段 $t = 1$, 不加 m , AdaLN-Zero 不以 w_{sc} 为条件。`sc_cfg` 为真时仅右段未知 denoise 槽再以 w_{sc} 为条件; 未知 decode 槽加 $m = \text{decode}$ 、不以 w_{sc} 为条件。`exit=decoder` 时 VAE-dec 读 token; $\mathcal{L}_{s1}$ 仍只用它做重建。 G 的注意力可见性按本族掩码节落地。时间梯子与各族如何铺 t , 见第 5 节。

4.3 日程与 checkpoint

Token 预算: 每族 $\times$ 每一 `latent_tune` 各有主训与扩展两份, 不共用 ($B_{\text{main}} / B_{\text{ext}}$ 见表)。切分长度与档比见附录 A。工程上主训与扩展是两次独立训练 (不同 `preprocess` / 不同 `checkpoint` 哈希), 扩展可用更大显存的卡。扩展启动时要求同参主训已正常结束, 并从主训 `checkpoint_latest` 的 EMA 初始化 live 权重 (不恢复优化器)。`mid` 的解冻点落在主训内; 若主训已过解冻点, 扩展立即保持解冻。

新 run 自 step 0 调用 `load_latent_artifact`, 扩展 run 再被主训 EMA 覆盖。此后只使用本 run 的 `checkpoint` 中的 latent 副本。无论是否训练 latent, 该 `checkpoint` 都必须包含完整 latent 参数。同一 Stage 续训先读取本 run, 再按累计 token 判断是否已超过 `latent_thaw_tokens`。

模型类字段进入模型指纹（包括 `latent_model+tag`、`sc_cfg`）；训练类进入训练指纹（包括 `latent_tune`）。现网只有一份训练哈希 `build_train_fingerprint`：整份 model YAML（含 `sampling`）与 `generate_cfg` 一并纳入；推理字段因此也会换 run 目录。规格分类上的「推理不进训练指纹」与此现网行为分开：本次不改全局管线。

4.4 共性参数

下表为 100m 规模默认值。修改默认值须写入对应指纹。 G 与优化器内部见附录。入口 `encoder / decoder` 只记 `latent_model` 与 `tag`。

名称	符号	释义	默认
模型（共用）			
<code>latent_model</code>		交给 <code>load_latent_artifact</code> 的家族名。	必填
<code>tag</code>		交给加载器的 tag。	必填
<code>n_embd</code>	D	G 隐层宽。与流空间 X 无关。	768
<code>latent_dim</code>	X	流空间维；须等于 artifact 输出维。	32
<code>max_seq_len</code>		位置编码与生成长度上限。	4096
<code>exit</code>		<code>decoder</code> : $\hat{x}_0 \in X$ 再 VAE-dec (Cola)； <code>linear</code> : 隐状态 $D \rightarrow V$ (ELF)。	<code>decoder</code>
训练（共用）			
<code>latent_tune</code>		加载之后是否训练 latent 本体。三档对应三个训练配置哈希。	<code>mid</code>
<code>latent_thaw_tokens</code>		仅 <code>mid</code> 的解冻点。	15B
<code>target_tokens</code>	B_{main}	主训预算（每族 $\times$ 每一 <code>latent_tune</code> ）。	45B
<code>ext_tokens</code>	B_{ext}	扩展预算（每族 $\times$ 每一 <code>latent_tune</code> ）。	5B

全局推理参数见第 9 节。CFG 见第 6 节。时间调度参数见第 5 节。

消融。 对照按下列网格执行。`latent_tune` 三档为主网格：每族 $\times$ 每一档各走主训与扩展预算（见表），不计入短训消融。其余训练短训不占用主跑 token 预算，建议每条使用扩展预算（切分见附录 A）。`ce_detach_g` 不列入训练消融。各表按模型 / 训练 / 推理分栏。全局推理消融见第 9 节。

名称	符号	默认	对照
模型			
<code>exit</code>		<code>decoder</code>	<code>linear</code>
训练			
<code>latent_tune</code>		<code>mid</code>	<code>frozen</code> 、 <code>full</code> （主网格；每档各走 $B_{\text{main}} + B_{\text{ext}}$ ）
<code>x0_source</code>		<code>z</code>	<code>mu</code>

5 时间调度

两族共用同一把确定梯子 $\{L_i\}$ 。档数 T (`time_step`) 由本族给出。不另设调度器开关。共性给出 Q 与 $\{L_i\}$ ；BELF / RELF 两小节给出如何把梯子铺到块或窗。

5.1 共性

设 $Z \sim \mathcal{N}(0, 1)$ 。 Φ 是它的累积分布函数

$$\Phi(z) = P(Z \leq z) = \int_{-\infty}^z \frac{1}{\sqrt{2\pi}} e^{-u^2/2} du,$$

Φ^{-1} 是分位函数：对 $p \in (0, 1)$ 有 $\Phi(\Phi^{-1}(p)) = p$ 。实现为 $\Phi^{-1}(p) = \sqrt{2} \operatorname{erfinv}(2p - 1)$ 。 $p \rightarrow 0^+$ 时 $\Phi^{-1}(p) \rightarrow -\infty$ ， $p \rightarrow 1^-$ 时 $\rightarrow +\infty$ ，故端点必须特判。

训练时间取 $t = \sigma(P_m + P_s Z)$ ，即 $\operatorname{logit}(t) \sim \mathcal{N}(P_m, P_s^2)$ ，其中 $\sigma(u) = (1 + e^{-u})^{-1}$ 。该分布的分位函数为

$$Q(p) = \begin{cases} 0 & p = 0, \\ \sigma(P_m + P_s \Phi^{-1}(p)) & 0 < p < 1, \\ 1 - \varepsilon & p = 1. \end{cases}$$

开区间结果再夹到 $[0, 1 - \varepsilon]$ 。 P_m, P_s, ε 见本节参数表。

档位在**概率轴**上等分，不是在 t 上等间隔。**time_step** T 是每块 / 每窗的 G 次数 ($T - 1$ 次流 + 1 次 decode; RELF 仍 $S \cdot T = W$)。须 $T \geq 4$ ，否则启动报错拒绝。概率轴按 $T - 1$ 均分：

$$L_i = Q\left(\frac{i}{T-1}\right), \quad i = 0, 1, \dots, T-1.$$

因而 $L_0 = 0$ ， $L_{T-1} = Q(1) = 1 - \varepsilon$ 。流的起点是 $L_0, \dots, L_{T-2}$ ；第 i 跳 ($i = 0, \dots, T-2$) $\Delta t = L_{i+1} - L_i$ ，故末次流 Euler 到 $1 - \varepsilon$ 。decode 也在 $t = L_{T-1} = 1 - \varepsilon$ ，不再往更干净走。没有第 T 个 Q 分位。训练与推理共用该确定梯子：推理不再从 logit-normal 随机采样，也不使用 $\operatorname{linspace}(0, 1)$ 。 $t = 1 - \varepsilon$ 上没有以该 t 为起点的 v-MSE G 。

名称	符号	释义	默认
模型（共用）			
denoiser_p_mean	P_m	$\operatorname{logit}(t) \sim \mathcal{N}(P_m, P_s^2)$ 的均值。	-1.5
denoiser_p_std	P_s	同上，标准差。	0.8
t_clean_eps	ε	$Q(1) = L_{T-1} = 1 - \varepsilon$ ；末流终点与 decode 的 t 。	0.05

5.2 BELF

train_t_schedule=block。梯子按上一小节： $L_i = Q(i/(T-1))$ ， $i = 0, \dots, T-1$ 。每次前向采样一跳 $i \sim \operatorname{Unif}\{0, \dots, T-1\}$ ，将该跳 $t = L_i$ **广播到当前块的未知槽**。 $i = 0, \dots, T-2$ ：未知槽 $m = \text{denoise}$ ，只监督该跳流的信噪比（v-MSE； $g = 1$ 时可按 CFG 修正 v_{tgt} ），以 w_{sc} 为条件，无 CE。 $i = T-1$ ：未知槽 $t = 1 - \varepsilon$ 、 $m = \text{decode}$ ，纯 CE，无 v-MSE、不以 w_{sc} 为条件；训练该跳 sc 为 0。已知槽钉 $t = 1$ ，不加 m 、不加噪。主跑 $W = T$ 。切块对齐 $0, W, 2W, \dots$ 。已提交的完整块进入 KV，流仅作用于当前块的未知槽。推理 **block_generate** 共 T 次 G ：跳 $i = 0, \dots, T-2$ 未知槽 $t = L_i$ ，Euler 步长 $\Delta t = L_{i+1} - L_i$ （末流走到 $1 - \varepsilon$ ）；跳 $T-1$ 在 $t = 1 - \varepsilon$ 再跑一次 G （ $m = \text{decode}$ ），**不 Euler**，直接读 token。已知槽每跳钉 $t = 1$ 并覆写为 encoder 干净码。混合块如何自生成、如何训，见第 7.1 节。

5.3 RELF

`train_t_schedule=mixed`。不设 `p_preroll` / `p_freeroll` / `p_postroll`，也不按 Cat 抽支。梯子按共性小节： $L_i = Q(i/(T-1))$ ， $i = 0, \dots, T-1$ 。时间场以档为单元：每档 S 槽共享同一 L_i （窗内铺全部 $L_0, \dots, L_{T-1}$ ）。右侧未知档每步只升一档（接到更高的 L_i ，最左的右侧档接到 $1-\varepsilon$ ）；最左档已是 $1-\varepsilon$ ，本步不 Euler。每次前向只指定当前窗的一帧 t 场。

先构造整窗 freeroll 场 F ：位置 $k = 0, \dots, W-1$ ，

$$F_k = L_{T-1-\lfloor k/S \rfloor}.$$

即档 $r = 0, \dots, T-1$ 占槽 $[rS, (r+1)S)$ ，值为 L_{T-1-r} 。最左档 $t = 1-\varepsilon$ (decode)，右组纯噪 L_0 。这把 F 是唯一模板。

截断看 BOS / EOS 落在虚拟满窗的哪一格，不是先抽档数再把 F 左右搬去凑满 W 。记 BOS、EOS 的文档下标为 i_{bos} 、 i_{eos} （均含该 token）。PAD 不是边界。窗起点 u 按文档起点 S 格对齐，覆盖 $[u, u+W)$ 。对窗内格 $k = 0, \dots, W-1$ ，文档下标 $j = u + k$ ：

- $j < i_{\text{bos}}$ ：BOS 之前，整段丢掉，不在当前窗。
- $j > i_{\text{eos}}$ ：EOS 之后，整段丢掉，不在当前窗。
- 否则：真槽， $t_k = F_k$ （沿用满窗 F 在该格上的值）。

丢掉的位置不赋 $t = 0$ 、不赋 $1-\varepsilon$ 平台，也不把留下的梯子挪到另一头去凑满 W 。真窗可以短于 W 。

两种切法相互独立、可以叠加，不是互斥分区。对虚拟满窗 F ：

- 句首切 (preroll)： $j < i_{\text{bos}}$ 的格丢掉。BOS 落在窗内格 $k_{\text{bos}} = i_{\text{bos}} - u$ ，其左全部截掉。
- 句尾切 (postroll)： $j > i_{\text{eos}}$ 的格丢掉。EOS 落在窗内格 $k_{\text{eos}} = i_{\text{eos}} - u$ ，其右全部截掉。

两端都不切 ($u \geq i_{\text{bos}}$ 且 $u+W-1 \leq i_{\text{eos}}$) 则整窗 $t \leftarrow F$ （句中 / freeroll）。只切左留下 F 的右侧（更噪）；只切右留下 F 的左侧（更净）；都切留下 F 的中段。损失按切完后的格计算，见第 8.3 节。

抽窗须使交集非空。做了句首切则真窗含 BOS；做了句尾切则真窗含 EOS。禁止为凑档把 BOS / EOS 挪出真槽；禁止把 PAD 当截断对象。BOS / EOS 不必落在档界上；所在档被截成短档时，留下的槽仍用该档的 F_k 。

禁止：将 token 窗对齐铺满为从 BOS 起的 W 、把最噪档搬到窗左、右侧铺 $t = 0$ 「尚未进入」；禁止把 F 左侧截掉再把剩余滑到窗右。

窗内时间场锁死为梯子 (`window_t=ladder`)，不进消融。不对齐前缀的余数走左切爬梯（与 preroll 同构），不是钉在满窗 F 左侧一次 CE；句首切不抽 r 。见第 8.1 节。

6 CFG

CFG 含两根相互独立的轴。 w_{sc} 仅进入右段未知 **denoise** 槽 AdaLN；decode 列不以 w_{sc} 为条件。推理为单次前向。 w_{ctx} 在训练时以概率丢弃前缀；推理默认仅运行带前缀的一次前向，外推时另需无前缀支路，共两次前向。出口 decoder 只读取 G 给出的 $\hat{x}_0$ ；CE 施加于哪些位置仍由本族规定。按槽拆损失之后，CFG 只作用在还要继续流的槽上。

6.1 w_{sc} : self-cond CFG

条件对象为上一跳的 $\hat{x}_0$ 。启用时 w_{sc} 进入 AdaLN 的 ScaleEmbedder, 与 t 的 TimestepEmbedder 分开。推理为单次前向。

启动开关。 `sc_cfg` 默认 `true`, 进入模型指纹, 因其决定网络是否包含 ScaleEmbedder 与 self-cond 通道。为真时必须具备 sc 通道。训练启动时若设为 `false`: 不构建 ScaleEmbedder、无 sc 通道、AdaLN 不以 w_{sc} 为条件 (仍逐列加 m)、 $v_{tgt} = v_z$ 、不做 teacher 的 Δv 前向; 推理忽略 w_{sc} 。`sc_cfg` 为真则 `self_cond=true`。

训练采样。 `sc_cfg` 为真时, 每个样本独立采样两项:

1. w_{sc} : 对齐 ELF, 在 logit-normal 上采样后再仿射到给定区间, 而非在该区间上均匀采样。先采样

$$z \sim \mathcal{N}(P_m^{sc}, (P_s^{sc})^2), \quad u = \sigma(z),$$

再按 ELF `sample_cfg_scale` 映到区间:

$$a = 1 + w_{\min}^{sc}, \quad b = 1 + w_{\max}^{sc}, \quad w_{sc} = a(b/a)^u - 1.$$

P_m^{sc}, P_s^{sc} 见表; $w_{\min}^{sc}, w_{\max}^{sc}$ 见附录。该分布既不是 $\text{Unif}[w_{\min}^{sc}, w_{\max}^{sc}]$, 也不是对 w_{sc} 本身的 LogUnif 。

2. $g \sim \text{Bern}(\text{sc_guided_prob})$: 是否按 CFG 修正速度目标, 以及是否将 uncond 的 $\hat{x}_0$ 写入 sc 通道。

$g = 0$ 时仍采样 w_{sc} 并写入右段 **denoise** 未知槽 AdaLN, 使推理所用的 w_{sc} 落在训练边缘分布内。decode 列始终不以 w_{sc} 为条件。`sc_cfg` 为假时不采样这两项。

启用时的一次 step。 本步若全是 decode 列 (BELF $i = T - 1$), 不跑 teacher、不修正 v_{tgt} , 学生只算 CE。否则三次前向使用同一个已采样的 w_{sc} (两个 teacher `no_grad`, 一个学生), 但 w_{sc} 只写入右段 **denoise** 未知槽 AdaLN:

1. Teacher uncond: self-cond 通道全 0 $\rightarrow \hat{x}_0^u, v_u$ 。
2. Teacher cond: 通道 $\text{sg}(\hat{x}_0^u) \rightarrow v_c$ 。
3. 学生: denoise 列对 v_{tgt} 计算 v-MSE; decode 列只计算 CE。

$$v_{tgt} = \begin{cases} v_z + (1 - 1/w_{sc})(v_c - v_u) & g = 1, \\ v_z & g = 0. \end{cases}$$

CFG 对 v_{tgt} 的修正仅施加于 denoise 列。 $g = 1$ 时学生的 sc 通道为 $\text{sg}(\hat{x}_0^u)$ (仍只写 denoise 列); $g = 0$ 时通道为 0 (denoise 未知槽 AdaLN 仍以已采样的 w_{sc} 为条件)。左段、已知槽与 decode 列 AdaLN 不以 w_{sc} 为条件。提交与 `forward_clean` 同左段。Teacher 的 Δv 对目标 stop-grad。

BELF: 流跳未知槽共享同一套 sc ; decode 跳训练 sc 为 0。已知槽 sc 为 0。RELF: 即使 $g = 1$, 窗内最右 S 个新噪声槽的 sc 仍为 0 (尚无上一跳估计); 最左 decode 档不以 w_{sc} 为条件。左段与干净槽的 sc 为 0。

推理:denoise 未知槽 sc 为上一步提交的 $\hat{x}_0$ (序列开始或新块/新槽/已知槽为 0); 仅 **denoise 未知槽** AdaLN 以推理 w_{sc} 为条件。BELF 末跳 / RELF 最左档: $m = \text{decode}$, $w_{sc} = 0$, sc 为上一跳流的 $sg(\hat{x}_0)$ 。前缀 KV 不随 w_{sc} 重算。

6.2 w_{ctx} : 上下文 CFG

条件对象为 2L 左段 / KV 前缀, 与 w_{sc} 独立, 不进入 AdaLN。

训练采样。 每个样本再独立采样 $d \sim \text{Bern}(p_{\text{drop}}^{\text{ctx}})$ (与 g 、 w_{sc} 均独立)。 $d = 1$ 时丢弃 2L 左段, 仅训练当前块/窗。此为训练期的无条件上下文。

推理默认仅运行带前缀的一次前向。若对 w_{ctx} 做外推: 无条件支路为**无前缀、仅运行当前块/窗**。然后

$$v = v_u^{\text{ctx}} + w_{ctx}(v_c^{\text{ctx}} - v_u^{\text{ctx}}).$$

消融取值见表。

6.3 CFG 参数

名称	符号	释义	默认
模型			
<code>sc_cfg</code>		是否启用整套 w_{sc} (含 sc 通道)。为假则无通道、不构建 ScaleEmbedder、不以 CFG 修正 v_{tgt} 。进入模型指纹。	true
训练			
<code>self_cond_cfg_p_mean</code>	P_m^{sc}	训练 w_{sc} 的 logit-normal 位置。对齐 ELF 时间场的 P_m 。	-1.5
<code>self_cond_cfg_p_std</code>	P_s^{sc}	尺度。	0.8
<code>sc_guided_prob</code>	p_g^{sc}	仅 <code>sc_cfg</code> : 本 step 按 CFG 修正速度目标并写入 sc 通道的概率。	0.5
<code>ctx_p_drop</code>	$p_{\text{drop}}^{\text{ctx}}$	丢弃 2L 左段的概率。与 g 独立。不进消融。	0.1
推理			
<code>w_sc</code>	w_{sc}	推理尺度。 <code>sc_cfg=false</code> 的 ckpt 忽略。YAML / <code>sampling</code> 键名即此; 实现仍接受 ELF 别名 <code>self_cond_cfg_scale</code> , 但已写 <code>w_sc</code> 时别名不生效。	3.0
<code>w_ctx</code>	w_{ctx}	推理上下文外推; 1= 不外推。YAML 键名即此; 别名 <code>ctx_cfg_scale</code> 同上。	1.0

训练 w_{sc} 的 logit-normal 参数见表; 采样区间 $w_{\text{min}}^{\text{sc}}, w_{\text{max}}^{\text{sc}}$ 见附录。推理 w_{ctx} 允许区间 $w_{\text{min}}^{\text{ctx}}, w_{\text{max}}^{\text{ctx}}$ 见附录。推理尺度可在同一 checkpoint 上更改, 不进入训练指纹。仅 `sc_cfg=true` 训练得到的 run 才对推理 w_{sc} 做扫描; 前缀 KV 不随 w_{sc} 重算。`sc_cfg=false` 的 run 不对 w_{sc} 扫描。全局提交、采样器与温度见第 9 节。

6.4 CFG 消融

对照按下列网格执行。`sc_cfg` 须单独短训；推理项使用同一 checkpoint。按槽拆损失已确定，见第 10 节，不作为消融项。`ctx_p_drop` 不进消融。

名称	符号	默认	对照
模型			
<code>sc_cfg</code>		<code>true</code>	<code>false</code> (训练启动时关闭整套 w_{sc} ，无 sc 通道)
推理			
<code>w_sc</code>	w_{sc}	3	{0.5, 1, 2, 3}; 仅 <code>sc_cfg=true</code>
<code>w_ctx</code>	w_{ctx}	1	{1, 2, 3, 5, 7}

7 BELF

块半自回归。联合分布写成 $p(z_0) = \prod_b p_G(z_0^{(b)} | z_0^{(<b)})$ 。每一块是一条条件 rectified flow：未知槽共享同一标量 t ；已知余数钉 $t = 1$ 。去噪块长等于本规格 `block_size` (W): 100m 默认 16，主跑 $W = T$ 。加载入口块长须 $\in \{1, W\}$ ；可训档另要求入口块长为 1。入口 encoder 仍按其加载块长做块因果，不随 W 改写。若训练均匀采样跳号 $i \sim \text{Unif}\{0, \dots, T-1\}$ ，则满未知块上每次前向施加 CE 的期望 token 数为 W/T 。对照 $W = 1$ 时 T 仍见表，不强制 $T = 1$ 。若将 W 与 T 分开设置，须写入指纹，并声明该期望变为 W/T 。当前块内已知余数如何自生成、如何训，见第 7.1 节。

7.1 Clean

主跑 `cond_mode=clean`。本节规定两件事：**自生成**（推理时已知余数与已提交前缀从哪来）和**训练**（如何抽到同构的混合块）。损失与注意力见第 7.3 节。

格子。 切块对齐 $0, W, 2W, \dots$ 。前缀长 L 时 $r = L \bmod W$ 。完整块 $\lfloor L/W \rfloor$ 进入 KV；当前块起点 $\lfloor L/W \rfloor \cdot W$ 。末块可短只指序列末端不够一整块。 $r = 0$ 时当前块没有已知余数。 $W = 1$ 时 r 恒为 0，`clean` 不触发、不抽混合块。

自生成。 `block_generate` 给定前缀 token（长 L ）：

1. **已齐的完整块**写入 KV。同一次生成循环里刚写完的块默认 `commit_x0hat=true`：把 G 的 $\hat{x}_0$ 提交，不再 encode。若这次调用的前缀是外部文本（提示，或上一轮已经写出再当作新前缀），完整块走 encoder 再按第 9 节提交。这是左段干净条件，不是当前块内的余数。
2. **当前块已知余数**只在 $r > 0$ 时出现：槽 $[0, r)$ 钉 $t = 1$ ，码是前缀对应位置的 encoder 干净码，不进入损失。这 r 个 token 可以是提示，也可以是上一轮已写出、这次当新前缀的自生成 token。 $r = 0$ 则整块未知。
3. 未知槽从噪声走 $T - 1$ 次流 + 1 次 decode。每跳之后把已知槽覆写回 encoder 干净码并钉 $t = 1$ ，避免 Euler / SDE 冲掉已知位。

4. 出口读整块 token，丢掉已知槽对应位置，只保留新写的。再把本块 $\hat{x}_0$ 提交进 KV。此后下一块整块未知：自生成的完整块只出现在左段 KV，不再以块内余数出现。
5. 一块一次读出。不会在同一块内把刚 decode 的 token 立刻改成已知余数再继续流。EOS 停。

训练。 以概率 `clean_block_prob` 把当前块抽成混合块：有效长 W' （末块可短），切点 $r \sim \text{Unif}\{1, \dots, W' - 1\}$ 。槽 $[0, r)$ 为已知 ($t = 1$ 、不进损失)， $[r, W')$ 为未知（走该跳 $t = L_i$ ）。其余前向整块均为未知。 $W = 1$ 或 $W' \leq 1$ 时不抽混合块。已知槽始终是 encoder 码，不用 $\hat{x}_0$ 冒充余数。右段 unknown Q 不可见同块 PAD K。可训档硬拒入口块长 $\neq 1$ ，不必二次 encode。仅冻结档且入口块长 = W 时，余数再 encode 一份条件句（当前块 $[r, W)$ 写成 PAD），已知覆写与 PAD 干净码用这份；插值靶仍来自整句 encode。左段 KV 训练时也是 encoder 干净码（教师强制）；推理左段默认是自生成的 $\hat{x}_0$ 。

排除项。 不随机挪块界。任意位置填空（左右都有干净上下文）须改 2L 可见性，不在本文档范围。

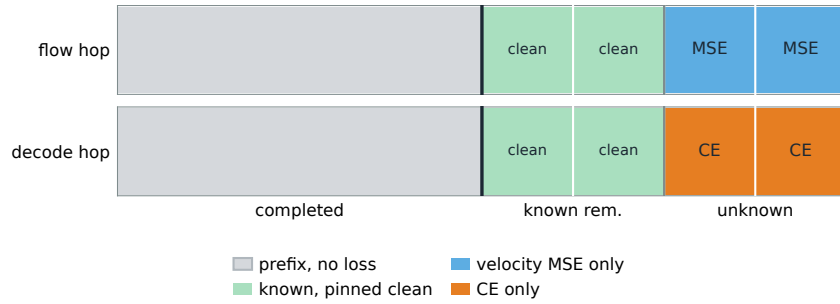


图 1: BELF clean: 当前块左侧为已知余数，钉干净、不进损失；右侧未知槽走块内同一 t 。

续写见图 2：前缀对不齐块界时，先在当前块未知槽上走完流与 decode、只读出新词并提交整块；此后下一块整块未知。

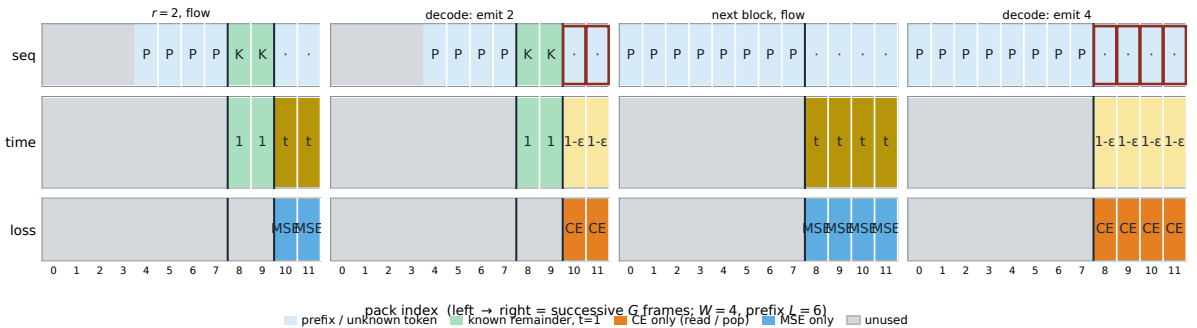


图 2: BELF clean 续写 ($W = 4$ ，前缀 $L = 6$ 故 $r = 2$)。从左到右为相继的 G 帧。先在混合块未知槽上流、再 decode 读出 2 个新词；提交后下一块整块未知。绿格为块内已知余数 ($t = 1$ ，不进损失)。流跳只画一帧（未知槽同一 t ）。红框为该帧读出的新词。

7.2 训练

加载并按 `latent_tune` 编码，按 W 切块（主训切分见附录 A），映射后 2L pack。CFG 采样见第 6 节。切块对齐 $0, W, 2W, \dots$ 。训练要求序列长度被 W 整除，否则报错拒绝；混合块余数按满块 W 抽取 $r \sim \text{Unif}\{1, \dots, W-1\}$ ，不按短末块有效长 W' 。推理末块可短，见第 7 节推理。是否混合块、已知槽如何钉干净，见第 7.1 节。按第 5.2 节指定右段时间并加噪：未知槽 $t = L_i$ ；已知槽见 Clean 节。

未知槽 $z_t = tx_0 + (1-t)\varepsilon$ 。 $i = 0, \dots, T-2$ ：右段未知有效位计算 v-MSE ($m = \text{denoise}$)， $\mathcal{L}_{\text{ce}} = 0$ 。 $i = T-1$ ：右段未知有效位纯 CE ($t = 1 - \varepsilon$, $m = \text{decode}$)，无 v-MSE、不以 w_{sc} 为条件。已知槽不进入 $\mathcal{L}$ 。不引入 m_d/m_c 。注意力与 sc 通道按第 7.3 节排布。默认 CE 对 G 反向传播。

接口：`forward(tokens)` 计算一块或一批块的混合损失。配置 `train_t_schedule=block`。生成使用 `block_generate`。混合块见第 7.1 节。

7.3 掩码排布

一次前向的 pack 是 [左段前缀 | 右段当前块]。切块对齐 $0, W, 2W, \dots$ 。一批块时右段按块拼接，各块独立套用下列规则，禁止跨块右段相互可见。训练序列长度须被 W 整除，因而训练右段没有短末块。推理末块可短于 W （序列末端不够一整块）。pad 不计入损失。左段与已知槽不进入 $\mathcal{L}$ 。损失为右段未知有效位平均，见训练节；不引入 m_d/m_c 。

注意力。 见图 3。已完成块写入 KV：块内双向、块间单向（后块可见前块）。当前块内双向，且可见文档位置早于本块起点的已完成内容；已完成块不能看当前块。一批右段时各块独立，禁止跨块互看。 $p_{\text{drop}}^{\text{ctx}}$ 时切断左段。

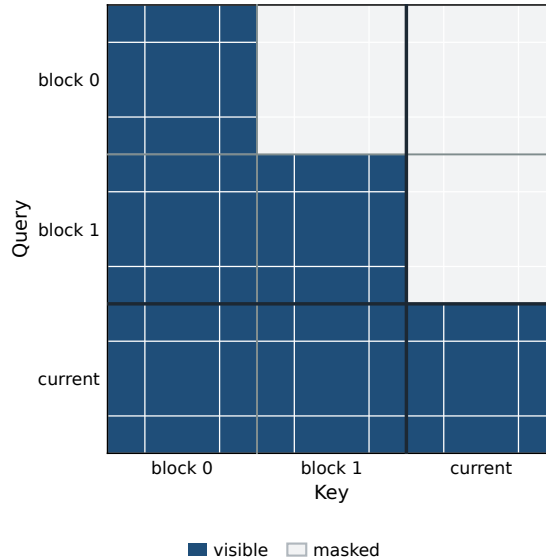


图 3: BELF 一次前向的注意力。查询看行、键看列。

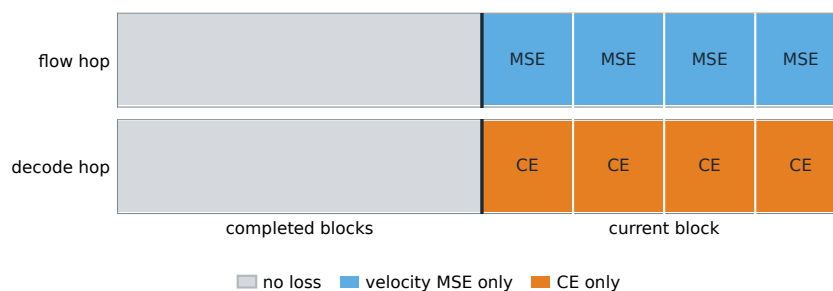


图 4: BELF: 已完成的前块不算损失。流跳只算速度 MSE; 最后一跳只算 CE, 不算 MSE。

self-cond 通道。 未知槽共享同一套 sc 。新块起始与 decode 跳训练为 0; 推理流跳使用上一跳 $\hat{x}_0$; 推理 decode 跳 sc 为上一跳流的 $sg(\hat{x}_0)$, $w_{sc} = 0$ 。左段与已知槽恒为 0。 p_{drop}^{ctx} 只改变注意力。已知槽 AdaLN 仅以 $t = 1$ 为条件, 不加 m 、不以 w_{sc} 为条件。未知 denoise 槽以 w_{sc} 为条件; decode 槽不以 w_{sc} 为条件。

7.4 推理

block_generate。 前缀如何切成「完整块进 KV + 当前块左侧已知余数」、每跳如何覆写已知槽、读出后如何提交, 见第 7.1 节。未知槽共 T 次 G : 跳 $i = 0, \dots, T - 2$ 未知槽 $t = L_i$, Euler $\Delta t = L_{i+1} - L_i$ (末流到 $1 - \varepsilon$); 跳 $T - 1$ 在 $t = 1 - \varepsilon$ 、 $m = \text{decode}$, **不 Euler**, 出口读 token。只要仍需续写且总长未到 `max_seq_len`, 后续块整块未知 (末块可短)。可因 EOS 提前停止。`sc_cfg` 为真时: 流跳未知槽 sc 使用上一步 $\hat{x}_0$, AdaLN 以推理 w_{sc} 为条件; decode 跳 $w_{sc} = 0$, sc 为上一跳流的 $sg(\hat{x}_0)$; 新块起始与已知槽为 0。采样器见第 9 节。

7.5 BELF 参数

名称	符号	释义	默认
模型			
<code>block_size</code>	W	G 去噪块长。默认见表 (100m 为 16); 入口块长独立, 须 $\in \{1, W\}$; 可训档另要求入口块长为 1。	16
<code>time_step</code>	T	每块跳数。须 $T \geq 4$, 否则报错。主跑 $W = T$ 。	16
训练			
<code>train_t_schedule</code>		训练期时间采样规则。	block
<code>cond_mode</code>		当前块内已知余数如何条件化。	clean
<code>clean_block_prob</code>	p_{clean}	训练时当前块抽成混合块的概率。不进消融。	0.05
训练长度		forward 要求序列长度被 W 整除, 否则报错。切分档须满足此约束。	须整除

CE 仅在 $i = T - 1$ 对未知有效位叠加 (该跳无 v-MSE、不以 w_{sc} 为条件); 不引入 m_d/m_c 。注意力见第 7.3 节。未知槽同一标量 t ; 已知槽钉 $t = 1$ 。生成使用 `block_generate`。加载入口块长须 $\in \{1, W\}$ 。训练序列长度须被 W 整除。默认 W 见表 (100m 为 16), 主跑 $W = T$; 满未知块上每次前向施加 CE 的期望 token 数为 W/T 。`clean_block_prob` 不进消融。全局提交、采样器与温度见第 9 节; CFG 尺度见第 6 节。

7.6 BELF 消融

对照按下列网格执行。全局推理消融见第 9 节。未知槽同一 t 、已知槽钉 $t = 1$ 、CE 仅最后一跳对未知槽（该跳无 MSE）为方法规则，不列入本表。`clean_block_prob` 不进消融。蒸馏方式待定，见第 10 节。

名称	符号	默认	对照
模型			
<code>block_size</code>	W	16	1（去噪块长对照）
<code>time_step</code>	T	16	{4, 8, 32}（固定 W ；从零训；期望 CE 的 token 数变为 W/T ）
训练			
<code>distill_time_step</code>	T'	（关）	{8, 4}（教师 <code>time_step</code> =16；学生少步； W 不变。蒸馏方式待定）

蒸馏与从零训练少步须分开报告。蒸馏方式待定，见第 10 节。主跑不蒸馏。`block_size` 去噪块长对照 $W = 1$ ；不再扫 {4, 8, 32}。`clean_block_prob` 不进消融。

8 RELF

滚动半自回归。窗内位置 k 具有局部时间 t_k ：

$$z^k = t_k z_0^k + (1 - t_k) \varepsilon^k.$$

主跑下每档 S 个槽共享同一 L_i ，而非逐槽独立采样时间。加载入口块长必须为 1（逐 token 因果）。RELF 不声明 `block_size`。窗长为 `window_size` (W)，与入口块长无关。须 $T \geq 4$ 且 $S \cdot T = W$ ，否则启动报错拒绝。当前窗已知余数如何自生成、如何训，见第 8.1 节。

生成分为三段。Freeroll 为滚动阶段：整窗铺 F ，梯子有 T 档，每档 S 个槽，最左档 $t = 1 - \varepsilon$ (decode)、右侧纯噪。每步一次 G ：最左档纯 CE、不 Euler；右侧未知档升一档后再 pop 这 S 槽，右端补入 S 个新噪声，回到 freeroll。 S 只表示档宽与 pop 个数。Preroll / postroll 把同一把 F 铺在虚拟满窗上：**BOS 之前整段截掉、EOS 之后整段截掉**；两切可叠加。截断位置由 BOS / EOS 所在格决定。总长上限为 `max_seq_len`。

8.1 Clean

主跑 `cond_mode=clean`。本节写窗内已知余数如何自生成、如何训。窗的 t 场（BOS 前 / EOS 后截 F ）见第 5.3 节；损失见第 8.3 节。

格子。 从文档起点按 S 对齐。前缀长 L （含 BOS）时 $r = L \bmod S$ ， $g = \lfloor L/S \rfloor \cdot S$ 。完整 pop $[0, g)$ 进入 KV。 $S = 1$ 时 r 恒为 0，不触发余数爬梯。不从任意 L 另开窗离开 S 格。

自生成。 `rolling_generate`。新词一律从 L_0 爬到 $1 - \varepsilon$ 才 CE / pop；余数只钉干净，不让新词跳过梯子。

- $L = 0$: preroll。虚拟窗 $u < i_{\text{bos}}$, BOS 落在窗内、**其左截掉**; u 每次右移 S , 直到 BOS 落到 $k = 0$ 再 CE 并进入 freeroll。clean 不介入。见图 9。
- $L > 0$ 且 $r = 0$ 且**已有** z_{carry} : 跳过 preroll 与爬梯。当前窗整窗未知, $t \leftarrow F$, 进入 freeroll。
- $L > 0$ 且 ($r > 0$ 或**尚无** z_{carry}): **余数爬梯** (与 preroll 同构)。对齐前缀冷启动 $r = 0$ 时 known 为空、 $n_{\text{write}} = S$ 。余数是文档 $[g, g+r)$ 。共 T 帧, hop $h = 0, \dots, T-1$:
 - 虚拟下标: 余数起点 $k_0 = W - S(h+1) = S(T-h-1)$ 。窗原点 $u = g - k_0$ 。 $k < k_0$ 的格丢掉 (不在当前窗; 已写完整 pop 在 KV)。
 - 槽 $[k_0, k_0+r)$ 为已知 (encoder 干净码, $t = 1$, 不进损失)。其余真槽未知, 铺各格 F_k (F 不滑)。句尾切: EOS 之后仍丢掉。
 - 每帧一次 G 。 $h < T-1$: decode 档仍在 k_0 左侧, 已截掉, $\sum m_c = 0$, 只 MSE; 未知档 Euler 升一档, 右端补 S 个 L_0 新噪声; **不 pop**; 已知槽覆写 $t = 1$ 。
 - $h = T-1$: $k_0 = 0$, 即混合满窗 F 。最左档不 Euler; 未知 decode 格 ($S-r$ 个) 纯 CE; pop 这 S 槽 (已知 r 来自前缀 encoder, 新写 $S-r$ 默认 $\hat{x}_0$)。随后 freeroll, 窗内不再带余数。
- BOS 若落在已知余数里: 钉干净, 不当未知, 也不踢出窗。
- 同一次滚动里刚 pop 的槽走 `commit_x0hat`; 若下次调用把已写文本当新前缀, 完整 pop 先 encode 再提交。

训练。 先取合法窗起点, 再按 BOS 前 / EOS 后截 F (须保住 BOS / EOS, PAD 不当截断对象)。 r 是截断生成: 已知为已经写出的真 token; 切点必须在 EOS / PAD 之前, 且至少留一个未知真槽; r 不得 $\geq S$ 。

- **不做句首切**: 以概率 `clean_block_prob` 抽**余数爬梯的一帧** (不是只抽末帧)。先抽 $h \sim \text{Unif}\{0, \dots, T-1\}$, 再铺该 hop 的左切: $k_0 = S(T-h-1)$ 。句首切判定为 $\text{bos_cut} = (u + k_0) < i_{\text{bos}}$ (留下的第一格是否仍在 BOS 前), 不是 $u < i_{\text{bos}}$ 。记该帧切完后真窗长 N' (从 k_0 起到 EOS / 右端)。令 $r_{\text{hi}} = \min(S-1, N'-1)$ 。若 $r_{\text{hi}} \geq 1$, 则 $r \sim \text{Unif}\{1, \dots, r_{\text{hi}}\}$; 已知 $[k_0, k_0+r)$ (encoder 干净码、 $t = 1$), 其余真槽未知 (各格 F_k , F 不滑)。 $h = T-1$ 时 $k_0 = 0$, 即混合满窗 F 。 $h < T-1$ 时 decode 档已切掉, 只监督 MSE。句尾切: EOS 不当已知。 $N' = 1$ 或 $S = 1$ 则不抽。
- **做了句首切** (留下的格仍有 $j < i_{\text{bos}}$): 不抽 r (那是 preroll, BOS 自己爬梯)。不把切掉的左侧改铺 $t = 0$, 也不在更噪的真窗左端再钉干净。两端都切 (含句首切) 同此。

不为凑档截掉 BOS / EOS 或把 PAD 当截断对象。已知槽与左段训练时都是 encoder 码 (教师强制); 推理左段默认是自生成的 $\hat{x}_0$ 。

排除项。 任意位置填空（左右都有干净上下文）不在本文档范围。禁止把余数钉在满窗 F 左侧、一次 G 就对 decode 档 CE（新词须走完 $L_0 \rightarrow 1 - \varepsilon$ ）。

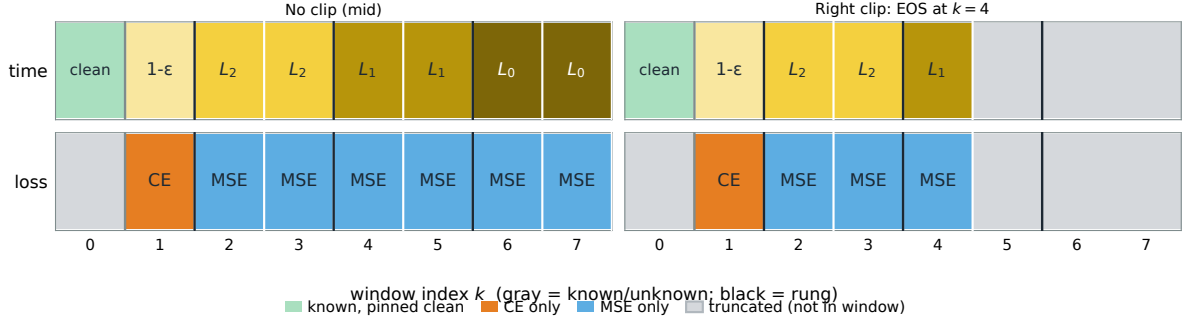


图 5: RELF clean 末帧 ($h = T - 1$): 余数已在虚拟 $k = 0$, decode 才 CE。左列句中满窗; 右列句尾切 (EOS 后截掉, F 不滑)。这是爬梯的最后一帧, 不是续写全过程。

续写全过程见图 6: 这 T 帧就是 $L > 0, r > 0$ 的推理。余数 K 从虚拟窗右端进入、随 hop 左移 S ; 其右第一个新词沿 F 从 L_0 升到 $1 - \varepsilon$, 先 MSE、到 decode 才 CE。不画随后的 freeroll。

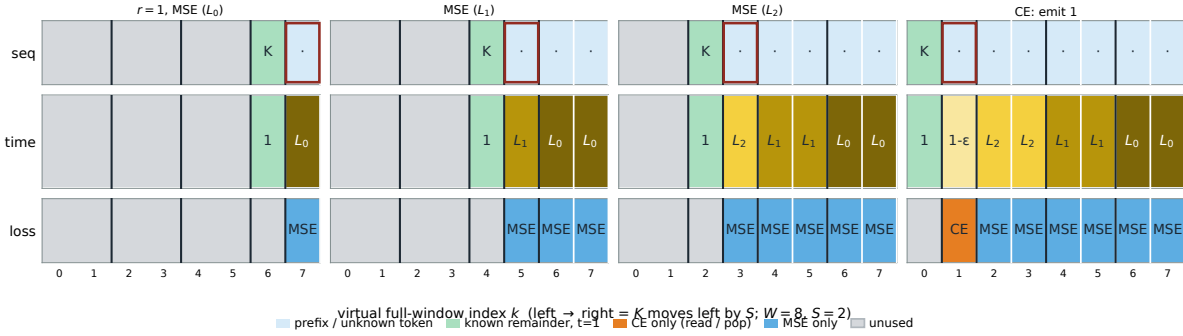


图 6: RELF clean 续写 ($W = 8, S = 2, r = 1$)。虚拟满窗左切: 绿格 K 第一帧在 $k=6$ (靠右), 之后每次左移 S 。红框为即将读出的新词: 先在 L_0, L_1, L_2 上打 MSE, 到 $1 - \varepsilon$ 才 CE。末帧 K 在 $k=0$, 即混合窗 F , 对应图 5。不画第一次 pop 之后的 freeroll。

8.2 训练

加载并按 `latent_tune` 编码, 映射后 2L pack。CFG 采样见第 6 节 (guided 时最右 S 槽 `sc` 仍为 0)。启动时若加载入口块长 $\neq 1$, 或 $T < 4$, 或 $S \cdot T \neq W$, 则报错拒绝。按第 5.3 节指定窗内时间: 先取合法窗起点, 再按 BOS 前 / EOS 后截 F 。是否抽余数爬梯的一帧 (未做句首切), 见第 8.1 节。

各槽按所赋 t_k 独立加噪; 已知槽不加噪。损失与注意力按第 8.3 节排布。默认 CE 对 G 反向传播。

接口: `forward(tokens)` 按当前窗相对序列起止截断铺 t 场, 再计算混合损失。配置为 `train_t_schedule=mixed`。生成使用 `rolling_generate`。混合窗见第 8.1 节。

8.3 掩码排布

一次前向的 pack 是 [左段前缀 | 右段当前窗]。窗长 W ，从 0 对齐。一批窗时右段按窗拼接，各窗独立套用下列规则，禁止跨窗右段相互可见。pad 位两项损失均为 0，留在 pack 里。左段与已知槽不进入 $\mathcal{L}$ 。须 $S \cdot T = W$ 。

注意力。 见图 7。与 BELF 同形的是「组内双向、组间单向」：已完成前段的组是加载入口块长（必须为 1，即逐 token 因果）；当前窗的组是档 S （同一档双向，右档可见左档，左档不可见右档）。窗可见文档位置早于本窗起点的已完成前段；前段不能看窗。一批右段时各窗独立，禁止跨窗互看。 $p_{\text{drop}}^{\text{ctx}}$ 时切断左段。

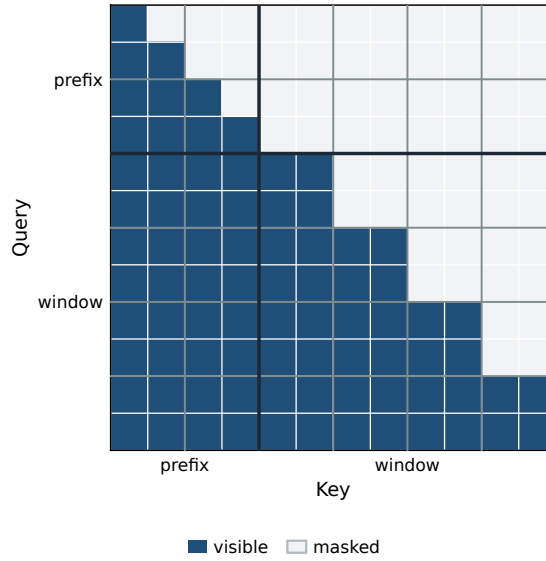


图 7: RELF 一次前向的注意力。查询看行、键看列。

损失 m_d/m_c 。 仅施加于右段。先按第 5.3 节把 F 切完，再对仍在窗的未知真槽按格赋值。不是按句中 / 句首 / 句尾查表；两种切法叠在同一窗上仍用同一式。已知槽、PAD、以及 BOS 前 / EOS 后丢掉的格均为 0。两项各自按掩码平均后再以 λ 相加：

$$\mathcal{L} = \lambda_{\text{mse}} \frac{\sum m_d \|\hat{v} - v_{\text{tgt}}\|^2}{\sum m_d} + \lambda_{\text{ce}} \frac{\sum m_c \text{CE}}{\sum m_c}.$$

对未知真槽 k ：

$$m_c(k) = \mathbf{1}[F_k = 1 - \varepsilon], \quad m_d(k) = \mathbf{1}[F_k < 1 - \varepsilon].$$

因而 $m_c = 1$ 则 $m_d = 0$ 。 $\sum m_c = 0$ 时 CE 项为 0（decode 档已全部切掉）； $\sum m_d = 0$ 时 MSE 项为 0。余数爬梯： $h < T - 1$ 时 decode 档在 k_0 左侧、已切掉，故 $\sum m_c = 0$ 、只 MSE；末帧 $k_0 = 0$ 时已知槽仍为 0， m_c 仅施加于未知的那 $S - r$ 格。

上式覆盖组合：只切左且 decode 档全在 BOS 前则 $m_c = 0$ ；BOS 落在 decode 档内则留下的 $1 - \varepsilon$ 未知格仍计算 CE；只切右且剩余未知都在 decode 档内则剩余未知纯 CE（已知不进损失）。余数爬梯各 hop 仍用同一式： m_c 仅施加于 $F_k = 1 - \varepsilon$ 的未知格。

未知真槽： $F_k = 1 - \varepsilon$ 则 $m = \text{decode}$ ，否则 $m = \text{denoise}$ 。丢掉的格不加 m 。

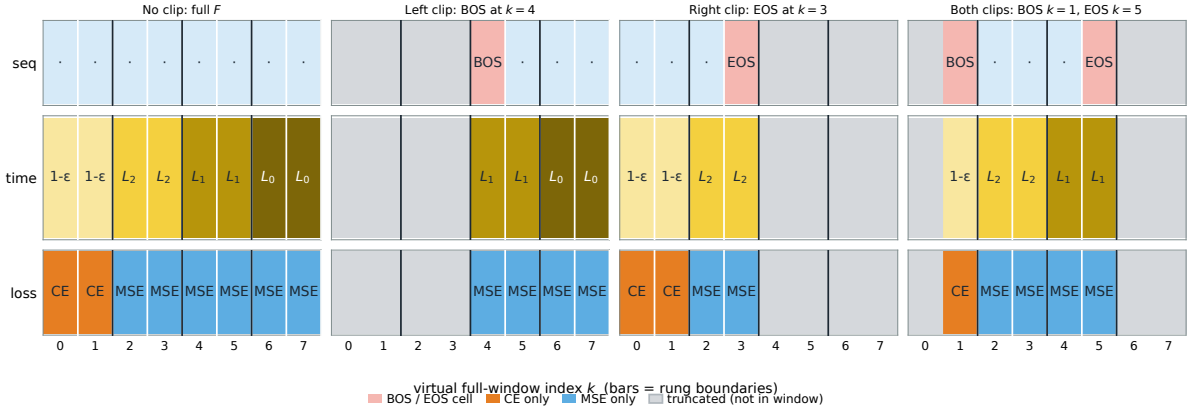


图 8: RELF 画的是虚拟满窗 F ($W = 8$, 每档 2 槽)。黄净、黑噪。橙 = 只算 CE; 蓝 = 只算 MSE。灰格是 BOS 之前或 EOS 之后, 不在窗、不赋 $t = 0$ 。前三列是只切一侧或都不切; 右列两端都切。损失按切完后的 F_k 按格计算, 不是三选一。

self-cond 通道。 右段已有上一跳估计的 denoise 未知槽写 $\text{sg}(\hat{x}_0^u)$ (guided 时)。最右新进的 S 个噪声槽 sc 恒为 0。最左 decode 档不以 w_{sc} 为条件; 推理时该档 sc 为上一跳流的 $\text{sg}(\hat{x}_0)$ 、 $w_{\text{sc}} = 0$ 。左段 / 已知槽 / 干净槽为 0。 $p_{\text{drop}}^{\text{ctx}}$ 只改注意力, 不改 m_d/m_c 。已知槽 AdaLN 仅以 $t = 1$ 为条件, 不加 m 、不以 w_{sc} 为条件。

8.4 推理

rolling_generate。 前缀如何切成「完整 pop 进 KV + 余数爬梯」、 $L = 0$ 是否 preroll、 $r > 0$ 时 T 帧后才第一次 pop, 见第 8.1 节。接近 `max_seq_len` 或更短目标时改 postroll: EOS 落在窗内, 其右全部截掉。PAD 不计、不当边界。EOS 只出现在句尾真槽里。首词与末词如何落到 decode 档再被读出, 见图 9。

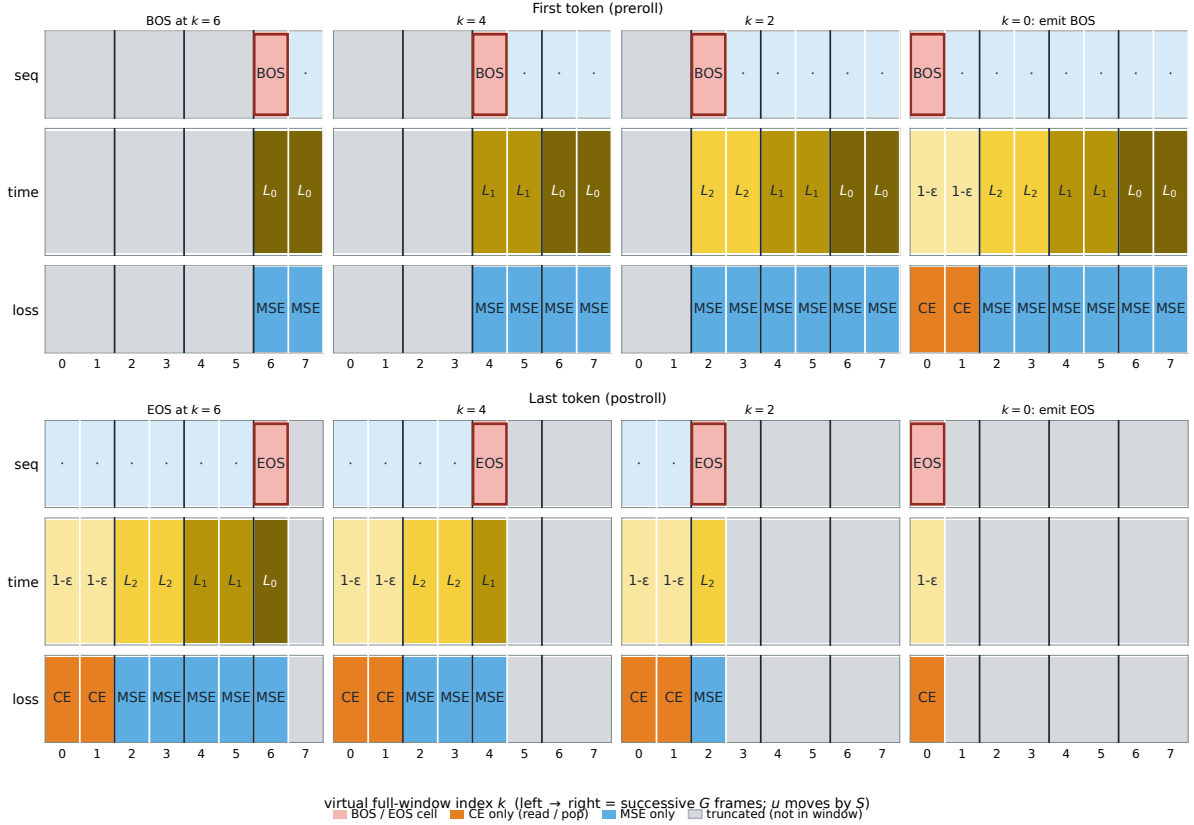


图 9: RELF 推理吐词 ($W = 8$, 每档 $S = 2$)。上行句首切: 窗起点 u 每次右移 S , BOS 在虚拟满窗上左移; 前三帧 decode 档仍在 BOS 之前 (已截掉), 无 CE; 末帧 BOS 落到 $k = 0$, 该档做 CE 并 pop 出首词。下行句尾切: EOS 之后丢掉、不再右补噪声; EOS 左移, 末帧只剩 $k = 0$ 的 EOS, CE 读出。灰格不在窗, 不赋 $t = 0$ 。

`sc_cfg` 为真时: denoise 未知槽 `sc` 使用上一步提交的 $\hat{x}_0$, AdaLN 以推理 w_{sc} 为条件; 最左 decode 档 $w_{sc} = 0$, `sc` 为上一跳流的 $sg(\hat{x}_0)$; 开局、新进噪声槽与已知槽为 0。采样器见第 9 节。

8.5 RELF 参数

名称	符号	释义	默认
模型			
<code>window_size</code>	W	滚动窗长。须 $S \cdot T = W$ 。	32
<code>time_step</code>	T	梯子档数。须 $T \geq 4$, 否则报错。须 $S \cdot T = W$ 。	16
<code>step_size</code>	S	每步从窗左端 pop 的槽数, 亦为档宽。须 $S \cdot T = W$ 。	2
<code>window_t</code>	t_k	窗内时间场。锁死 <code>ladder</code> , 不进消融。	<code>ladder</code>
训练			
<code>train_t_schedule</code>		训练期时间采样规则。	<code>mixed</code>
<code>cond_mode</code>		当前窗内已知余数如何条件化。	<code>clean</code>
<code>clean_block_prob</code>	p_{clean}	未做句首切时抽余数爬梯一帧的概率 (含 <code>hop h</code> 与 <code>r</code> ; 切点在 EOS 前)。不进消融。	0.05

生成使用 `rolling_generate`。窗的 t 场由句首切 / 句尾切对整窗 F 给出, 两切可叠加, 不

另设分支概率。余数爬梯见第 8.1 节。CE 只叠在 $F_k = 1 - \varepsilon$ 的未知槽，不进参数、不进消融。`window_t` 锁死 `ladder`，不进消融。`clean_block_prob` 不进消融。消融见下表。全局提交、采样器与温度见第 9 节；CFG 尺度见第 6 节。

8.6 RELF 消融

对照按下列网格执行。改 W 时须保持 $T \geq 4$ 且 $S \cdot T = W$ 。全局推理消融见第 9 节。CE 只叠 $F_k = 1 - \varepsilon$ 的未知槽，不进消融。`window_t` 锁死 `ladder`，不进消融。`clean_block_prob` 不进消融。蒸馏方式待定，见第 10 节。

名称	符号	默认	对照
模型			
<code>window_size</code>	W	32	改 W 时同步改 T 或 S (须 $S \cdot T = W$)
<code>step_size</code>	S	2	1 (须 $T = W/S$)
训练			
<code>distill_time_step</code>	T'	(关)	{8,4} (教师 <code>time_step</code> =16; 学生少步; <code>window_size</code> 不变, 须 $S \cdot T' = W$ 。蒸馏方式待定)

蒸馏与从零训练少步须分开报告。蒸馏方式待定，见第 10 节。主跑不蒸馏。改 T' 时 `window_size` 不变，则同步改 `step_size` 以保持 $S \cdot T' = W$ 。

9 推理

本节给出两族共用的推理项。这些字段可在同一 checkpoint 上更改，不进入训练指纹。BELF 生成使用 `block_generate`，RELF 生成使用 `rolling_generate`。CFG 的推理尺度见第 6 节。

默认 `commit_x0hat=true`：将 G 的 $\hat{x}_0$ (流空间 X) 写入 G 的 KV。对照 `false`：先由出口读出 token，再 encode 采 z (`sample=(x0_source==z)`) 写入，空间与 `x0_source` 一致。外部前缀完整块仍走 `encoder`、`sample=false`。两条策略分开报告。训练时左段仍是 `encoder` 的干净码。出口无状态 (`decoder` 时 $\hat{x}_0 \in X$ 直接 VAE-dec)，不维护 KV。提交时左段 $t = 1$ ，AdaLN 不以 w_{sc} 为条件，`sc` 通道为 0。

采样默认 `sampling_method=sde` (γ 见表)，对照 `ode`。`sde` 时 `churn` 关在最后一次流 (BELF 跳 $T - 2$ ；RELF 升档侧)，`decode` 前向无速度。`sc_cfg` 为真时，仅 `denoise` 未知槽 AdaLN 以推理 w_{sc} 为条件 (仅开训 checkpoint 可扫描)；`decode` 列 $w_{sc} = 0$ 。前缀 KV 不随 w_{sc} 重算。

9.1 推理参数 (全局)

名称	符号	释义	默认
推理			
<code>commit_x0hat</code>		提交 $\hat{x}_0$ ；关则再 encode 采 z 。	<code>true</code>
<code>sampling_method</code>		<code>sde</code> 或 <code>ode</code> 。	<code>sde</code>
<code>sde_gamma</code>	γ	仅 <code>sde</code> 的噪声系数。	1.5
<code>temperature</code>		出口温度； ≤ 0 为 <code>argmax</code> 。	0.0

9.2 推理消融（全局，不重训）

对照按下列网格执行。同一 checkpoint。 w_{sc} / w_{ctx} 扫描见第 6 节。评测长度见附录 A。 `sde_gamma` 取值对齐 ELF 已报告的工作点：论文默认 1.0；32 步设定 1.5（本文档默认）；8/16 步设定 2.0。 $\gamma = 0$ 时 SDE 退化为 ODE，与 `sampling_method=ode` 等价，故消融不另列 $\gamma = 0$ 。

名称	符号	默认	对照
推理			
<code>commit_x0hat</code>		<code>true</code>	再 encode 采 z
<code>sampling_method</code>		<code>sde</code>	<code>ode</code>
<code>sde_gamma</code>	γ	1.5	{1.0, 1.5, 2.0}; 仅 <code>sde</code>

10 设计选择与未决项

前面各章给出已确定的做法。本章记录已闭合的设计选择，以及仍未确定的项。

10.1 已确定： w_{sc} 进入哪一段 AdaLN

左段是 $t = 1$ 的干净前缀。若 AdaLN 亦以推理 w_{sc} 为条件，前缀 KV 会随 w_{sc} 变化，与训练中「左段不以 w_{sc} 为条件」不一致。规格如下：**仅右段未知 denoise 槽以 $(t, w_{sc}, m = \text{denoise})$ 为条件**。未知 decode 槽以 $(t, m = \text{decode})$ 为条件、不以 w_{sc} 为条件。提交、左段与已知槽均不以 w_{sc} 为条件、不加 m 。 $g = 0$ 时 sc 仍为 0，但 denoise 未知槽 AdaLN 仍以已采样的 w_{sc} 为条件。扫描 w_{sc} 时不重算前缀 KV。见第 6 节、第 9 节。

10.2 已确定： BELF / RELF 用 clean 处理已知余数

主跑 `cond_mode=clean`。自生成与训练抽样见第 7.1 节、第 8.1 节，此处不复述。 RELF：未做句首切时抽余数爬梯的一帧 ($h \sim \text{Unif}\{0, \dots, T-1\}$ ，新词从 L_0 爬到 decode 才 CE)；句首切不抽 r 。禁止钉满窗 F 左侧一次 CE。任意位置填空不在本文档范围。已确定。

10.3 已确定：读出位纯 CE，CFG 仅施加于仍在流的槽

同一 $\hat{x}_0$ 上不同时施加「经 CFG 修正的流」与「真词 CE」。 G 仍一份（禁止 ELF `decoder_prob` 双模式）。按跳号 / 槽位拆损失：

- BELF：跳 $i = 0, \dots, T-2$ 未知槽只 v-MSE ($g = 1$ 时可按 CFG 修正 v_{tgt} 、以 w_{sc} 为条件)；跳 $i = T-1$ 在 $t = 1 - \varepsilon$ 纯 CE，无 MSE、不以 w_{sc} 为条件、不 Euler。
- RELF：仍在窗的 $F_k = 1 - \varepsilon$ 未知槽纯 CE ($m_c = 1, m_d = 0$)，不以 w_{sc} 为条件、不 Euler；其余未知真槽只 MSE + CFG，每步只升一档。两种切法可叠加，损失按格、不按三行表。
- AdaLN-Zero 逐列 (t, w_{sc}, m) 。 m 仅未知真槽。 BOS 前 / EOS 后不在窗，与满窗右侧 $L_0 = 0$ 的 denoise 真槽不是一回事。

推理 decode 列也不以 w_{sc} 为条件。教师强制缺口仍在（训练插值 z 与推理 Euler z ）。已确定。

10.4 未决：少步蒸馏

两族消融都列 `distill_time_step` {8,4} (教师 $T = 16$)，与从零训少步分开报告。主跑不蒸馏。蒸馏配方尚未确定：是否冻结已训教师、学生改哪些参数、损失如何书写，两族同一未决项。RELF 还须保持 $S \cdot T' = W$ 。

A 长度日程

切分仅用于构造训练样本。模型只声明 `max_seq_len=4096`。

- 主训对齐长度 512，全部样本为该长度。
- 扩展对齐长度 2048;5B 按档 token 比 1:2:3:4 分成 0.5+1.0+1.5+2.0 B, 档 {256, 512, 1024, 2048}。
- 训练切分止于 2048。2048 < $S \leq 4096$ 仅出现于推理外推。eval 主报 2048, 4096 作为外推。
- BELF 训练另要求每条样本长度被 W 整除 (100m 默认 $W = 16$; 上列档均满足)。不整除则 `forward` 报错。RELF 无此整除约束。

B 共性参数附表

下列为共性章未列入主表的字段，仍按模型 / 训练 / 推理分栏。CFG 主表见第 6 节，全局推理主表见第 9 节。修改这些字段仍须写入对应指纹（推理类除外），但本文档不以它们为对照轴。入口 encoder / decoder 只记 `latent_model` 与 `tag`；隐维、词表、VAE 容量、s1 系数以加载结果为准，不列入本表。出口只有一层线性，无层数键；`decoder` 再接的 VAE-dec 以加载结果为准。表中 100m 数字供该规模实现对齐，并非另一套规范。BELF 的 `block_size`、RELF 的窗字段在本族章，不在本表。

名称	符号	释义	默认
模型			
<code>proj_bias</code>		映射是否带偏置。	<code>true</code>
<code>proj_norm</code>		映射后是否 RMSNorm。	<code>rmsnorm</code>
<code>whiten</code>		映射前逐维仿射标准化。	<code>true</code>
<code>whiten_on</code>		白化统计对象。	<code>mu</code>
<code>n_layer</code>		AdaLN-Zero Transformer 层数。	12
<code>n_head</code>		头数。须整除 <code>n_embd</code> 。	12
<code>mlp_ratio</code>		SwiGLU 扩展比。	4.0
<code>dropout</code>		注意力 / FFN dropout。	0.0
<code>qk_norm</code>		Q/K 每头 RMSNorm。	<code>true</code>
<code>rope_theta</code>	θ	RoPE 基频。	10000
<code>t_freq_dim</code>		正弦时间嵌入频率维。	256

名称	符号	释义	默认
ada_ln		主跑 AdaLN-Zero (无仿射 LN、残差 gate、true 调制末层与 FinalLayer 输出线性零初始化)。逐列 (t, w_{sc}, m) ; sc_cfg 为真时仅右段未知 denoise 槽再以 w_{sc} 为条件。decode 列加 $m = \text{decode}$ 、不以 w_{sc} 为条件。左段与已知槽 $t = 1$ 、不加 m 、不以 w_{sc} 为条件。不用 ELF in-context。不进消融。	
num_time_tokens		整句时间前缀个数。0= 条件只走 AdaLN-Zero。	0
right_attn		2L 右段内部注意力。	bidirectional
self_cond_mode		self-cond 通道接入方式。	concat ($2D \rightarrow D$)
num_self_cond_cfg_tokens		CFG prefix 个数。本规格为 0。	0
tie_lm_head		词表头是否绑 embedding。	false
lm_head_bias		$D \rightarrow V$ 是否带偏置。	false
noise_sigma	σ	加噪标准差。	1.0
训练			
self_cond_cfg_min	$w_{\min}^{\text{sc}}$	训练 w_{sc} 区间下界。	0.5
self_cond_cfg_max	$w_{\max}^{\text{sc}}$	训练 w_{sc} 区间上界。	5.0
vel_eps	ε_t	速度分母 $\max(1 - t, \varepsilon_t)$ 的下界。	10^{-3}
lambda_mse	λ_{mse}	v-MSE 系数。	1.0
lambda_ce	λ_{ce}	CE 系数。	1.0
ce_detach_g		出口 CE 是否在 $\hat{x}_0$ 处 stop-grad。默认对 G 反向传播; 不列入训练消融。	false
ctx_source		2L 左段用 z 还是 μ 。	z
x0_source		流的干净端 / v-MSE 靶。	z
optimizer.dtype		训练精度。与 ELF 配方相同。	bf16
learning_rate		AdamW 学习率。与 ELF 配方相同。	0.002
muon_learning_rate		Muon 学习率。与 ELF 配方相同。	0.002
weight_decay		weight decay。	0
muon_weight_decay		Muon weight decay。	0
beta1	β_1	AdamW β_1 。	0.9
beta2	β_2	AdamW β_2 。	0.95
grad_clip		全局梯度裁剪。	1.0
muon_momentum		Muon momentum。	0.95
muon_ns_steps		Muon Newton-Schulz 步数。	5
use_muon		隐藏层 Linear 用 Muon。	true
ema_decay		EMA; 0 关闭。	0.9999
warmup_ratio		预热占 max_steps 的比例。	0.005
min_lr_ratio		最小学习率相对峰值; 1=constant (BELF/RELF 脚本 --set)。	1.0

名称	符号	释义	默认
global_batch_size		全局批。s1 为 512; s2 用 curriculum global_seq_batch=128。	512 (s1)
推理			
ctx_cfg_min	$w_{\min}^{\text{ctx}}$	推理 w_{ctx} 下界。	1
ctx_cfg_max	$w_{\max}^{\text{ctx}}$	推理 w_{ctx} 上界。	7

References

- [1] ELF Authors. *ELF: Embedded Language Flows*. 2026. arXiv: [2605.10938 \[cs.CL\]](#).
- [2] ByteDance Seed. *Continuous Latent Diffusion Language Model*. 2026. arXiv: [2605.06548 \[cs.CL\]](#).
- [3] AURORA-LM Authors. *AURORA-LM: Autoencoding Unified Representation for Continuous-Latent Diffusion Language Modeling*. 2026. arXiv: [2608.02602 \[cs.CL\]](#).
- [4] CoDAR Authors. *CoDAR: Continuous Diffusion Language Models are More Powerful Than You Think*. 2026. arXiv: [2603.02547 \[cs.CL\]](#).
- [5] Marianne Arriola, Aaron Gokaslan, Tess Piriyaakulkij, et al. “Block Diffusion: Interpolating Between Autoregressive and Diffusion Language Models”. In: *International Conference on Learning Representations*. 2025. arXiv: [2503.09573 \[cs.CL\]](#).
- [6] David Ruhe et al. “Rolling Diffusion Models”. In: *International Conference on Machine Learning*. 2024. arXiv: [2402.09470 \[cs.CV\]](#).